Impulsively-Started Channel Flow PINN

Pre-Coding Notes

1 Physical Problem

A fluid at rest between two parallel plates (channel walls), subjected at $t = 0$ to a suddenly-applied constant pressure gradient. The flow starts uniform/at rest and relaxes over time into the steady parabolic (Poiseuille) profile.

Because the channel is straight and infinitely long (translationally invariant in x), the flow stays:

- Unidirectional: $v = 0$ everywhere
- Uniform in x: $\partial u/\partial x = 0$

This collapses the full 2D unsteady Navier–Stokes system down to a 1D diffusion equation with constant forcing — dramatically simpler than the general unsteady 2D case, while still being genuinely *unsteady* physics.

2 Governing Equation

Starting point (general unsteady 2D incompressible Navier–Stokes):

$$u_t + uu_x + vu_y = -\frac{1}{\rho}p_x + \nu\left(u_{xx} + u_{yy}\right) \tag{1}$$

$$v_t + uv_x + vv_y = -\frac{1}{\rho}p_y + \nu\left(v_{xx} + v_{yy}\right) \tag{2}$$

$$u_x + v_y = 0 \tag{3}$$

Reduces to (using $v = 0$, $\partial/\partial x = 0$ for this specific geometry):

$$u_t = G + \nu u_{yy} \tag{4}$$

where:

- u_t — local acceleration (this is the "unsteady" term)
- νu_{yy} — viscous diffusion across the channel
- G — constant forcing term (the imposed pressure gradient, suddenly switched on at $t = 0$)

Domain: $y \in [0, 1]$ (wall-to-wall, normalized channel height)
Initial condition: $u(y, 0) = 0$ (fluid at rest)
Boundary conditions: $u(0, t) = u(1, t) = 0$ (no-slip at both walls, for all t)

Forcing G is set so that the eventual steady-state solution matches the normalized Poiseuille profile:

$$u_{\text{steady}}(y) = 6y(1-y) \tag{5}$$

(mean velocity = 1, peak = 1.5 at centerline)

Non-dimensionalization: lengths by channel height H, velocities by a reference velocity, time by the diffusive time scale, giving dimensionless diffusive time:

$$\tau = \nu t \tag{6}$$

This is the "clock" the transient runs on — everything below is expressed in terms of τ.

3 Analytical Reference Solution

This is the ground truth used for validation.

Because both walls are no-slip, any valid $u(y)$ can be written as a sine series (sine automatically satisfies $u = 0$ at $y = 0, 1$).

3.1 Steady-state parabola's sine expansion

Only odd n survive, since the parabola is symmetric about the centerline and even sine modes are antisymmetric about that point:

$$6y(1-y) = \sum_{n \text{ odd}} \frac{48}{n^3\pi^3} \sin(n\pi y) \tag{7}$$

3.2 Full transient solution

Since $u(y, 0) = 0$, each mode simply grows from 0 toward its steady-state value at its own exponential rate:

$$u(y,\tau) = \sum_{n=1,3,5,\ldots}^{\infty} \frac{48}{n^3\pi^3} \sin(n\pi y) \left[1 - e^{-n^2\pi^2\tau}\right] \tag{8}$$

3.3 Sanity checks on the formula

- At $\tau = 0$: every bracket $= 0 \Rightarrow u = 0$ everywhere ✓ (matches initial condition)
- As $\tau \to \infty$: every bracket $\to 1 \Rightarrow$ the sum collapses to $6y(1-y)$ ✓ (matches steady Poiseuille, Eq. 5)

Practical note: higher modes decay much faster (rate $\propto n^2$), so truncating at $n = 1, 3, 5, 7, 9$ (5 terms) is already highly accurate for any τ of interest. This will be implemented as a plain NumPy function (no autograd, no network) — just a formula evaluated on a grid, used purely as the comparison reference.

4 PINN Construction

Inputs: (y, t) — note: *not* (x, y, t) — since the physics doesn't depend on x at all for this simplified geometry, x isn't even an input. This is a deliberate simplification relative to the general 2D unsteady Navier–Stokes case: it keeps the "unsteady" character of the problem while removing the extra spatial dimension and the pressure–velocity coupling (no p, no continuity equation, no nonlinear advection terms — the convective term vanished because $v = 0$ and $\partial u/\partial x = 0$).

Output: $u(y, t)$ — a single scalar, same as the heat equation cases.

Network architecture: same style feed-forward `tanh` MLP as before, e.g. $[2, 32, 32, 32, 1]$.

Physics residual (this is what autograd computes and penalizes):

$$\text{residual} = u_t - G - \nu u_{yy} \tag{9}$$

Loss terms:

$$\mathcal{L} = \mathcal{L}_{\text{physics}} + \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{walls}} \tag{10}$$

- $\mathcal{L}_{\text{physics}}$: residual ≈ 0 at scattered collocation points in (y, t)
- $\mathcal{L}_{\text{IC}}$: $u(y, 0) = 0$ at $t = 0$ points
- $\mathcal{L}_{\text{walls}}$: $u(0, t) = u(1, t) = 0$ at wall points, across all t

This is structurally almost identical to the 1D heat equation script — same number of loss terms, same derivative machinery (u_t, u_{yy} via autograd) — just with a constant forcing term G added to the PDE and different IC/BC values. This is intentional: it's the natural first "unsteady fluid" step up from what's already been built, without introducing the full complexity (pressure field, nonlinear advection, 2D spatial coupling) discussed for the general case.

Not needed for this simplified case (would be needed for the full 2D unsteady Navier–Stokes problem, but not here):

- Pressure field p as a network output
- Continuity equation loss
- Nonlinear advection terms (uu_x, vu_y)
- Causal time-weighting / curriculum training (helpful for harder unsteady problems, but this one is simple/well-behaved enough it likely won't be needed — can add later if training shows problems)

5 Validation & Visualization Plan

Ground truth: the analytical series solution from Section 3, evaluated on the same (y, t) grid as the PINN — no separate CFD solver needed.

Planned comparison plots (PINN and reference always shown together, never the PINN in isolation):

1. **Profile overlay:** $u(y)$ at several fixed times (e.g. early, mid, late τ), PINN (solid) vs. analytical (dashed), same axes.
2. **Centerline time-trace:** $u(y = 0.5, t)$ vs. t, PINN vs. analytical overlaid — shows the whole transient at a glance.

3. **Heatmap comparison:** $u(y,t)$ as a 2D field — PINN / analytical / absolute error, three panels side by side, shared color scale on the first two.

4. **Error-over-time curve:** mean/max absolute error vs. t, to check the PINN doesn't degrade at later times.

6 Summary of Simplifications vs. Full Unsteady 2D Navier–Stokes

Full problem	**This simplified version**
Inputs (x, y, t)	Inputs (y, t) only
Outputs (u, v, p)	Output u only
3 coupled PDEs (2 momentum + continuity)	1 PDE (1D diffusion + constant forcing)
Nonlinear advection terms	None (advection vanishes identically)
Pressure–velocity coupling + reference anchor	No pressure unknown at all
Likely needs causal/curriculum training	Standard training should suffice
No closed-form reference (needs numerical CFD)	Exact infinite-series reference available

Table 1: Simplifications made relative to the general unsteady 2D Navier–Stokes problem.

This version keeps the genuinely new ingredient — **time** — while stripping away the other sources of difficulty, so it's a clean, validated stepping stone before attempting the full coupled 2D unsteady case.

Some note to learn the basics

What "feed-forward tanh MLP" means, and why

- **MLP (Multi-Layer Perceptron)** — the most basic neural network shape: a stack of "fully connected" (or "dense") layers, where every neuron in one layer connects to every neuron in the next. In our code, this is literally the nn.Linear layers stacked in nn.Sequential.

- **Feed-forward** — information flows strictly one direction: input → hidden layers → output, no loops, no memory of previous inputs. This is the opposite of a recurrent network (RNN/LSTM), which feeds its own output back into itself over a sequence. For a PINN, there's no "sequence" in the usual sense — each (y, t) point is evaluated independently — so feed-forward is the natural (and simplest) choice.

- tanh — the activation function sitting between layers (see the nn.Tanh() calls in the code). Without an activation function, stacking linear layers would just collapse into one big linear layer, no matter how many you stack — the activation is what lets the network represent curved, nonlinear functions at all.

Why tanh **specifically, for PINNs:**

This is the one detail that's specific to PINNs, not just generic ML taste. Recall: the whole PINN mechanism depends on differentiating the network's output with respect to its inputs, often twice (e.g. u_yy in our channel-flow script). This means the activation function itself needs to be smooth and have well-behaved derivatives, all the way up.

- tanh is infinitely differentiable and smooth everywhere — perfect for this.
- **ReLU** (the default choice in most of ordinary deep learning, e.g. image classification) is piecewise linear — its second derivative is zero almost everywhere and undefined at the kink. If we'd used ReLU in our heat-equation or channel-flow scripts, the u_{xx} or u_{yy} terms in the physics residual would come out as zero (or garbage) almost everywhere, and the physics loss would be meaningless. This is a common beginner mistake in PINNs.
- **Sigmoid** also works smoothness-wise, but tends to saturate (flatten out) more aggressively than tanh, making gradients vanish faster during training —tanh is usually preferred in practice.

Other choices you'll encounter in PINN literature (this is where it gets more specialized/research):

- **Sinusoidal activations (SIREN)** — using sin(x) as the activation instead of tanh. These networks are very good at representing high-frequency, oscillatory solutions (useful for wave equations), something plain tanh networks can struggle with.
- **Swish/SiLU** — a smooth, modern activation (x · sigmoid(x)) sometimes used as a tanh alternative; smooth enough for PINNs, occasionally trains faster.
- **Adaptive activations** — a trick specific to PINN research (Jagtap et al.) where you add a trainable scaling parameter inside the activation function itself (e.g. tanh(a·x) where a is learned), letting the network adjust its own nonlinearity sharpness during training. Often improves convergence on hard problems.
- **Fourier feature networks** — instead of changing the activation, you preprocess the inputs by expanding them into a set of sine/cosine features ($\sin(\omega x), \cos(\omega x)$, various frequencies ω) before feeding them to an otherwise-normal tanh MLP. This helps a plain MLP represent fine spatial/temporal detail that it otherwise struggles to learn (a known weakness called "spectral bias").

Beyond the network architecture — variants of the PINN framework itself (different from just swapping the activation):

- **DeepONet/Fourier Neural Operators (FNO)** — instead of learning one specific solution u(x,t) for one specific setup, these learn an operator: a mapping from an entire input function (e.g., "any" initial condition, or any forcing profile) to the corresponding solution. This is a bigger conceptual jump — surrogate for a whole family of problems, not just one instance.
- **Variational PINNs (VPINN)/Conservative PINNs (cPINN)** — reformulate the physics loss using the equation's weak/integral form rather than the raw pointwise residual, which can be more numerically stable for certain PDEs (e.g. ones with shocks or discontinuities).
- **XPINN** — domain decomposition: split a complicated domain into subregions, train a separate (smaller) PINN in each, and stitch them together with matching conditions at the interfaces. Useful for very large/complex domains.

For toy problems like the heat equation or the unsteady flow treated here the tanhMLP is the suited PINN architecture. The fancier variants above are used in specfics situation where limitations (e.g. spectral bias, huge domains, or wanting to solve a family of problems instead of one instance) appear.

What the physics residual actually is, and what "penalizing" really means

Recall the governing equation, e.g. from the heat equation: $u_t - \alpha u_x x = 0$. This equation is a statement of truth — any function $u(x,t)$ that is a true physical solution must make this expression exactly zero, everywhere in the domain.

The **physics residual** is simply: take the network's current candidate function $u(x,t)$, plug it into that same expression, and see what number comes out.

$$residual = u_t - ALPHA * u_{xx}.$$

If the network were a perfect solution, residual would be exactly 0 everywhere. Early in training, when the network is basically a random function, the residual will be large and messy. As training proceeds, the residual should shrink toward zero across the whole domain.

Careful here, because there are two different notions of error, and it's important not to conflate them:

- **The residual** measures self-consistency with the known physical law — "does this candidate function obey the equation it's supposed to obey?" It requires no knowledge of the true solution at all — you can compute it even when you have no idea what the right answer looks like. This is exactly why PINNs are useful for problems where you don't have an analytical or numerical reference (unlike our toy examples, where we cheated by having one for validation).
- **The "error"** we computed in our scripts (e.g. $error = np.abs(u_pinn - u_ref)$) is a totally different quantity — the gap between the PINN's prediction and a known ground truth. This is only computable when a reference solution exists, and we used it purely to check our work, not as part of training at all. Notice: the analytical reference solution never appears anywhere in the training loop — only in the inference/validation section.

So: the residual is not "the unresolved physics" in the sense of a mistake relative to a known answer — it's a measure of physical inconsistency, computable in complete ignorance of the true answer. That's precisely what makes it trainable at all.

What "penalizing" concretely means (this is standard ML machinery, not PINN-specific):

In ordinary machine learning, a loss function is a single number that training tries to minimize. If you want the model to satisfy several different desirable properties at once, a very common approach is to build the loss as a **weighted sum of several terms**, one per property you care about:

$$total_loss = loss_{physics} + loss_{ic} + loss_{bc}$$

"Penalizing" the residual means adding a term to this sum that grows large whenever the residual is far from zero (here, $loss_physics = mean(residual)$, i.e. mean squared error against a target of zero). Since the optimizer's entire job is to make $total_loss$ as small as possible, and $loss_physics$ is one ingredient of that sum, gradient descent will naturally push the network's weights in directions that shrink the residual — alongside also shrinking the IC and BC mismatches at the same time.

Crucially — nothing is added to the network's output. There's no "correction term" bolted onto $u(x,t)$ after the fact. The word "penalty" refers entirely to how the loss used for training is constructed, not to any modification of the surrogate's predictions. The network itself only ever has one job: take (x,t) in, produce u out. The penalty lives purely in how we score that output during training, which shapes which weights the optimizer settles on.

This is exactly the same idea as, say, **L2 weight decay** in standard deep learning (a term added to the loss that penalizes large weight values, encouraging simpler models) or a **classification loss with a fairness penalty** (an extra loss term discouraging biased predictions). PINNs just introduce a new kind of penalty term — one built from a physical law via automatic differentiation — but the underlying "add a term to the loss, let gradient descent sort it out" mechanism is completely standard ML practice, not something unique to PINNs.

One related concept worth flagging, since you're new to this — an alternative to penalizing boundary/initial conditions (soft constraints) is to instead **build them directly into the network's architecture** as hard constraints — e.g., multiplying the network's raw output by $y(1-y)$ so that the boundary condition $u = 0$ at $y = 0, 1$ is satisfied automatically, by construction, no matter what the network weights are. We didn't do this in our scripts (we used the soft/penalty approach throughout, which is simpler to set up and more general), but it's a common refinement once you're comfortable with the basics — it removes the boundary loss term entirely and can make training more reliable, at the cost of some extra design work per problem.